



가천대학교 스마트시티학과 – 가상공간과 메타버스

Lecture 03 - Virtual City Modeling Lab

Chris Ohk

utilForever@gmail.com

- Lecture 2의 도시 모델을 실행 가능한 프로그램으로 만든다.
 - 격자와 객체를 연결하고, 입주와 재정의 변화를 계산한다.
- 6개의 함수를 작성하며 도시 시뮬레이션을 단계적으로 만든다.
 - 작성 → 단계 검사 → 화면 확인 → 조건을 바꿔 실험

- 압축을 푼 실습 폴더에서 실행한다.
- Windows: `py -m tkinter → py app.py`
- macOS / Linux: `python3 -m tkinter → python3 app.py`
- `model.py`를 저장한 뒤 앱을 닫고 다시 실행한다.

- 제공된 코드에서 TODO 주석 부분을 채운다.

파일	역할	수업 중 수정
app.py / gui.py	실행 / 지도 / 버튼 표시	제공
city_data.py	초기 도시 / 편집 / 저장	제공
model.py	도시를 계산하는 함수들	직접 작성
checks.py	단계별 자동 검사	실행

- 코드에 있는 '기본 도시'는 화요일의 5 × 4 예제다.
- 주택 A : 6/10명, 주택 B : 8/10명, 예산 : 100코인, 시작 일자 : 0일

화면 기능	조작 / 확인	연결되는 실습
조회	셀 / 건물의 상태 읽기	자료구조 확인
주택 / 공원	빈 땅에 클릭	TODO 3 - 배치
전체 인구	초기에는 미구현	TODO 2 - 합계
1일 진행	입주 / 예산 / 날짜 갱신	TODO 6 – 갱신

- 도시는 하나의 딕셔너리로 관리한다.
- grid는 위치, buildings는 객체, rules는 계산 규칙이다.

키	의미	기본값 / 예
grid	셀을 담은 2차원 리스트	4행 × 5열
buildings	ID로 찾는 객체 딕셔너리	A, B, P
day / budget	시간 / 잔액	0일 / 100코인
rules	입주 / 세금 / 유지비 규칙	하루 단위
external_power	외부 전력 공급	True

- 좌표는 (x, y) , 리스트 접근은 `grid[y][x]`다.
- x 는 오른쪽, y 는 아래쪽으로 증가한다.



```
cell = city["grid"][y][x]
cell["terrain"]      # "land" 또는 "water"
cell["road"]         # True 또는 False
cell["building_id"]  # None 또는 건물 ID

height = len(city["grid"])
width = len(city["grid"][0])
```

- 셀은 건물 ID를 저장하고, 객체 딕셔너리에서 내용을 찾는다.
- 셀의 ID와 객체의 좌표는 항상 서로 일치해야 한다.



```
ident = city["grid"][1][1]["building_id"]  
house = city["buildings"][ident]
```

```
house["kind"]           # "house"  
house["x"], house["y"]  # (1, 1)  
house["population"]     # 6  
house["capacity"]       # 10  
house["status"]         # "operating"
```

- `neighbors4(city, x, y)`를 작성한다.
 - 상하좌우 중 지도 안에 있는 좌표만 리스트로 반환한다.
 - 중심 셀과 대각선은 포함하지 않는다.
- $(0, 0)$ 의 이웃 : $(1, 0), (0, 1)$

- 방향 오프셋을 더한 다음 경계를 검사한다.



```
directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]  
result = []  
for dx, dy in directions:  
    nx, ny = x + dx, y + dy  
    # nx, ny가 지도 안에 있으면 result에 추가  
  
# result를 반환
```

- `model.py`의 TODO 1을 완성해 보자.
 - `python checks.py --stage 1`
 - 중앙 4개, 모서리 2개, 가장자리 3개를 확인한다.
- 오류를 찾을 때는 좌표와 경계 조건부터 확인한다.

- 범위를 검사한 좌표만 결과에 추가한다.



```
height, width = len(city["grid"]), len(city["grid"][0])
result = []

for dx, dy in [(-1, 0), (1, 0), (0, -1), (0, 1)]:
    nx, ny = x + dx, y + dy

    if 0 <= nx < width and 0 <= ny < height:
        result.append((nx, ny))

return result
```

실습 2 : 전체 인구

- `total_population(city)`를 작성한다.
 - 주택 객체의 `population`만 더해서 정수를 반환한다.
 - 기본 도시는 $6 + 8 = 14$ 명, 주택이 없으면 0명이다.
- 힌트 : `buildings.values()`를 순회한다.

직접 해보기 : 전체 인구

- `model.py`의 TODO 2를 완성해 보자.
 - `python checks.py --stage 2`
 - 앱을 다시 실행하고 전체 인구 14명을 확인한다.
- 빈 도시와 공원만 있는 도시에서는 무엇이 반환될까?

- 원본 객체에서 합계를 계산하면 중복 상태를 줄일 수 있다.



```
population = 0
```

```
for building in city["buildings"].values():  
    if building["kind"] == "house":  
        population += building["population"]
```

```
return population
```

실습 3 : 건물 배치

- `place_building(city, x, y, kind)`를 작성한다.
 - 0일의 빈 육지 셀에 주택 또는 공원을 배치한다.
 - 새 주택은 인구 0명, 수용량 10명, 운영 중 상태다.
 - ID는 H1, H2, ... / P1, P2, ... 중 가장 작은 빈 번호다.
- 도시를 수정한 뒤 새 건물 ID를 반환한다.

- 실패할 조건을 먼저 검사하고, 그 뒤에 상태를 바꾼다.
 - 제공 : 종류 / 날짜 / 좌표
 - 직접 작성 : 지형 / 도로 / 기존 건물
 - 불가능한 배치는 ValueError로 알린다.
- 오류가 발생했다면 도시는 배치 전과 같아야 한다.

- `model.py`의 TODO 3을 완성해 보자.
 - `python checks.py --stage 3`
 - 프로그램의 '주택' / '공원' 모드로 빈 셀에 배치한다.
- 물 / 도로 / 기존 건물에 배치하려는 경우 거절되어야 한다.

- 배치에 성공하는 경우 객체를 추가하고 셀 참조를 기록해야 한다.
 - 객체의 $(x, y) \leftrightarrow \text{grid}[y][x]$ 의 `building_id`
- 배치에 실패하는 경우 어떤 변경도 남기지 않는다.
 - 기본 도시에서 물 $(4, 0)$ 에 배치해도 건물 수는 그대로다.

실습 4 : 입주 조건

- `eligible(city, house)`는 입주 자격을 True/False로 반환한다.
 - 주택 여부 / 운영 상태 + 인접 도로 + 가까운 운영 중 공원
 - 외부 전력도 공급되어야 한다.
- 입주 가능 인원은 다음 실습에서 따로 계산한다.

- 도로는 이웃 셀, 공원은 객체 사전에서 찾는다.
 - $\text{공원 거리} = |\text{주택 } x - \text{공원 } x| + |\text{주택 } y - \text{공원 } y|$
- A(1, 1)와 P(2, 1)의 거리는 1이다.
 - 대각선으로 한 칸씩 떨어져 있으면 거리는 2다.

직접 해보기 : 입주 조건

- `model.py`의 TODO 4를 완성해 보자.
 - `python checks.py --stage 4`
 - 기본 도시의 A / B는 True, 공원 없는 도시는 False다.
- 전력을 끄거나 인접 도로를 지우면 결과가 달라져야 한다.

- 각 조건을 계산한 뒤 and로 결합한다.



```
if house["kind"] != "house":  
    return False  
if house["status"] != "operating":  
    return False  
if not city["external_power"]:  
    return False  
  
# 제공된 반복문에서 road_access와 park_access 계산  
return road_access and park_access
```

실습 5 : 입주 인원

- `arrivals(city, house)`는 오늘 입주할 인원을 반환한다.
 - 조건 미충족이면 0명, 충족이면 두 제한 중 작은 값이다.
 - `min(하루 최대 입주 인원, 수용량 - 현재 인구)`
- 인구 9/10명인 주택에는 최대 1명만 입주할 수 있다.

- `model.py`의 TODO 5를 완성해 보자.
 - `python checks.py --stage 5`
 - 6/10 → 2명, 9/10 → 1명, 10/10 → 0명
- 함수 호출 뒤에도 원래 주택 인구는 변하지 않아야 한다.

- 조건 검사와 용량 제한을 순서대로 적용한다.



```
if not eligible(city, house):  
    return 0
```

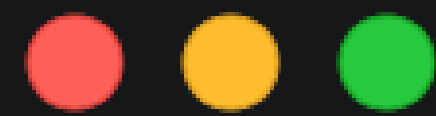
```
remaining = house["capacity"] - house["population"]  
return min(city["rules"]["growth_per_day"], remaining)
```

실습 6 : 하루 진행

- `step(old)`는 다음 날의 새 도시 상태를 반환한다.
 - `old`에서 판단하고, 복사한 `new`에만 기록한다.
 - 입주 / 재정 / 날짜를 갱신한 뒤 새 상태를 반환한다.
- 호출 후에도 `old`는 그대로 남아야 한다.

이전 상태와 다음 상태

- 하루 갱신은 이전 상태를 기준으로 계산한다.



```
new = deepcopy(old)
start_population = total_population(old)

for ident, house in old["buildings"].items():
    if house["kind"] != "house":
        continue
    incoming = arrivals(old, house)
    # new의 같은 ID 주택에 새 인구를 기록
```

- 오늘의 세금은 하루 시작 인구로 계산한다.
 - 다음 예산 = 현재 예산 + 시작 인구 $\times 3 - 10$
- 첫날 : $100 + 14 \times 3 - 10 = 132$ 코인
 - 입주 후 18명을 사용하면 144코인이 되어 규칙과 달라진다.

직접 해보기 : 하루 진행

- `model.py`의 TODO 6을 완성해 보자.
 - `python checks.py --stage 6`
- '기본 도시'에서 '1일 진행'을 두 번 눌러 보자.
 - 인구 : 14 → 18 → 20
 - 예산 : 100 → 132 → 176

- 입주 계산의 기준과 세금 계산의 기준을 고정한다.

시점	A	B	전체 인구	예산
0일	6	8	14명	100코인
1일	8	10	18명	132코인
2일	10	10	20명	176코인

실험 1 : 공원 제거

- 공원만 없앤 초기 도시와 기준 도시를 비교한다.
- '공원 없는 도시'를 선택하고 이틀을 진행한다.

시나리오	0일 : 인구 / 예산	1일 : 인구 / 예산	2일 : 인구 / 예산
공원 있음	14 / 100	18 / 132	20 / 176
공원 없음	14 / 100	14 / 132	14 / 164

실험 2 : 나만의 도시

- '빈 지도'에서 주택 / 공원 / 도로를 원하는 대로 배치한다.
 - 주택마다 입주 조건을 확인하고 이틀을 진행한다.
- 시작 상태로 돌아가 조건 하나만 바꾸어 비교한다.
 - 좌표 / 전력 / 공원 중 하나를 바꾸고 결과를 예측해 본다.

- 공간 조회 → 객체 배치 → 조건 판단 → 상태 갱신
 - 6개의 함수가 GUI 뒤의 도시 모델을 구성한다.
- `python checks.py`로 전체 결과를 확인해 보자.

감사합니다.

utilForever@gmail.com

<https://github.com/utilForever>

Facebook, Twitter, Instagram, LinkedIn: utilForever